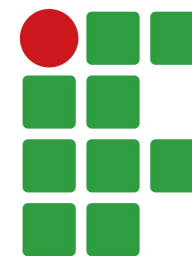


PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco

O projeto Android

- 2003: Andy Rubin, Rich Miner, Nick Sears e Chris White criam a Android Inc.
 - O objetivo era desenvolver um sistema operacional de código aberto para uma ampla variedade de dispositivos móveis.
- 2005: o Google adquire a Android Inc., dando início a uma nova fase no desenvolvimento da plataforma.
- 2007: surge o Bugdroid, o famoso robô verde.



O projeto Android

- 2007: Google anuncia a Open Handset Alliance (OHA), formada por empresas de tecnologia, operadoras, fabricantes e desenvolvedores.
 - A OHA tinha como objetivo promover padrões abertos e impulsionar o desenvolvimento da plataforma Android.



O projeto Android

- 2008: lançamento do HTC Dream (T-Mobile G1), o primeiro smartphone comercial com Android.



- 2009 — Android 1.5 (Cupcake): introdução do teclado virtual e dos primeiros grandes recursos que ajudaram a popularizar a plataforma.
- 2010 — Android 2.3 (Gingerbread): trouxe melhorias importantes de desempenho e recursos, tornando-se uma das versões mais difundidas.

O projeto Android

- 2011 — Android 3.0 (Honeycomb): versão desenvolvida especialmente para tablets.
- 2011 — Android 4.0 (Ice Cream Sandwich): unificação das versões para smartphones e tablets.
- 2014 — Android 5.0 (Lollipop): introdução do Material Design, que marcou uma grande mudança na interface visual do Android.
- 2015 — Android 6.0 (Marshmallow): introdução do modelo moderno de permissões em tempo de execução.

O projeto Android

- 2019 — Android 10: o Google abandona os nomes de sobremesas nas versões oficiais e passa a usar apenas números.
- 2026:



PRODUTO

Google lança Android 17 com mais IA e melhorias de desempenho

Anunciado no Google I/O, o novo Android 17 foi lançado oficialmente nesta terça-feira (16). Entre as promessas estão ainda mais recursos de IA e mais desempenho para smartphones.

Versões do Android



Android 1.0



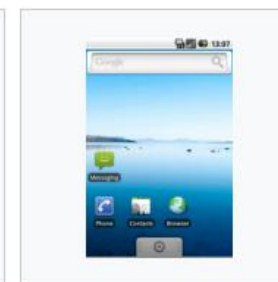
Android 1.1



Android 1.5 -
Cupcake



Android 1.6 - Donut



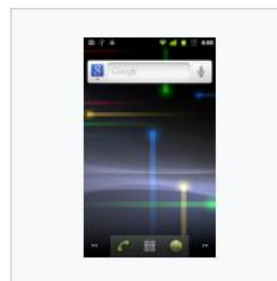
Android 2.0 - Eclair



Android 2.1 - Eclair



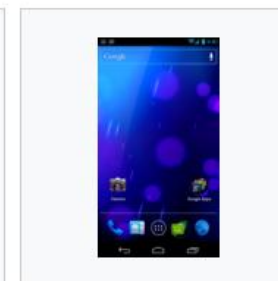
Android 2.2 - Froyo



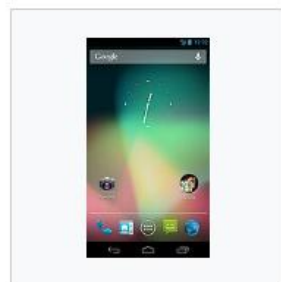
Android 2.3 -
Gingerbread



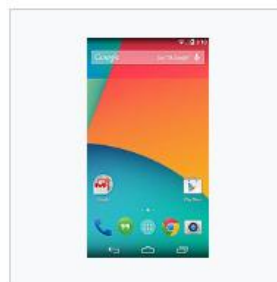
Android 3.0/3.1/3.2 -
Honeycomb



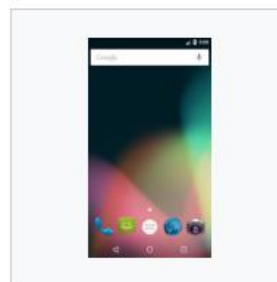
Android 4.0 - Ice
Cream Sandwich



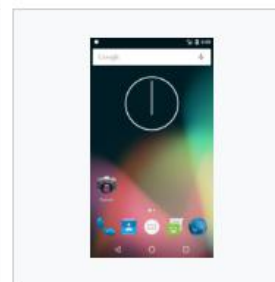
Android 4.1/4.2/4.3 -
Jelly Bean



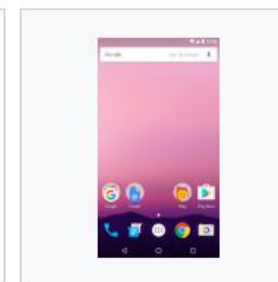
Android 4.4.2 - KitKat



Android 5.0 - Lollipop

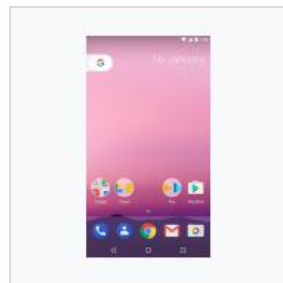


Android 6.0 -
Marshmallow

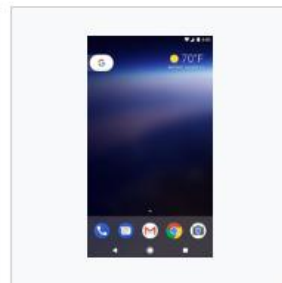


Android 7.0 - Nougat

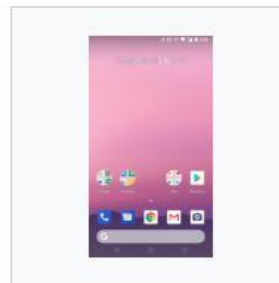
Versões do Android



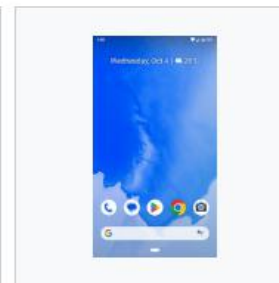
Android 7.1 - Nougat



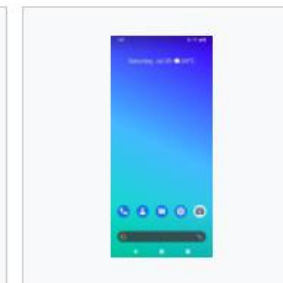
Android 8.0 - Oreo



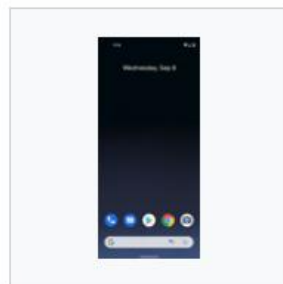
Android 8.1 - Oreo



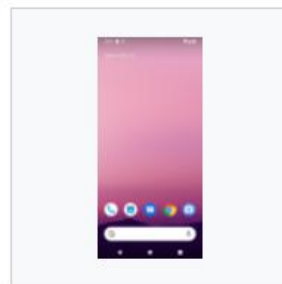
Android 9 - Pie



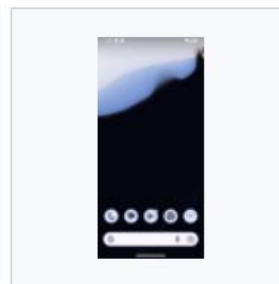
Android 10 - Q



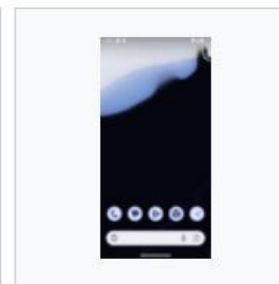
Android 11 - Red
Velvet Cake



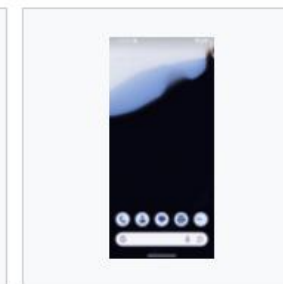
Android 12 - Snow
Cone



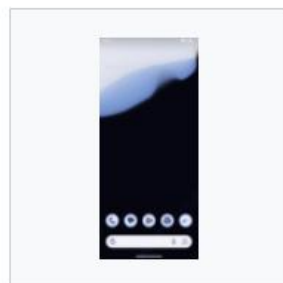
Android 13 - Tiramisu



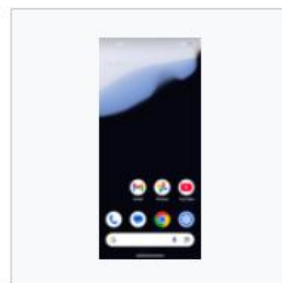
Android 14 - Upside
Down Cake



Android 15 - Vanilla
Ice Cream



Android 16 - Baklava



Android 17 -
Cinammon Bun

O SDK do Android

- O Android SDK (Software Development Kit) é o conjunto de ferramentas necessário para desenvolver, testar e depurar aplicações Android.
 - APIs e bibliotecas: recursos para desenvolver aplicações para diferentes versões do Android.
 - Ferramentas de desenvolvimento: compilação, depuração, análise e empacotamento dos aplicativos.
 - SDK Manager: permite instalar e atualizar plataformas Android, ferramentas e outros pacotes do SDK.
 - Device Manager: permite criar e gerenciar dispositivos virtuais (emuladores) para testar os aplicativos.
- Documentação: developer.android.com

API Level

- O API Level é um número inteiro utilizado para identificar o conjunto de APIs e recursos disponibilizados pelo Android.
 - Cada versão do Android possui um API Level de referência.
 - O API Level é utilizado para determinar a compatibilidade entre aplicativos e versões do Android.

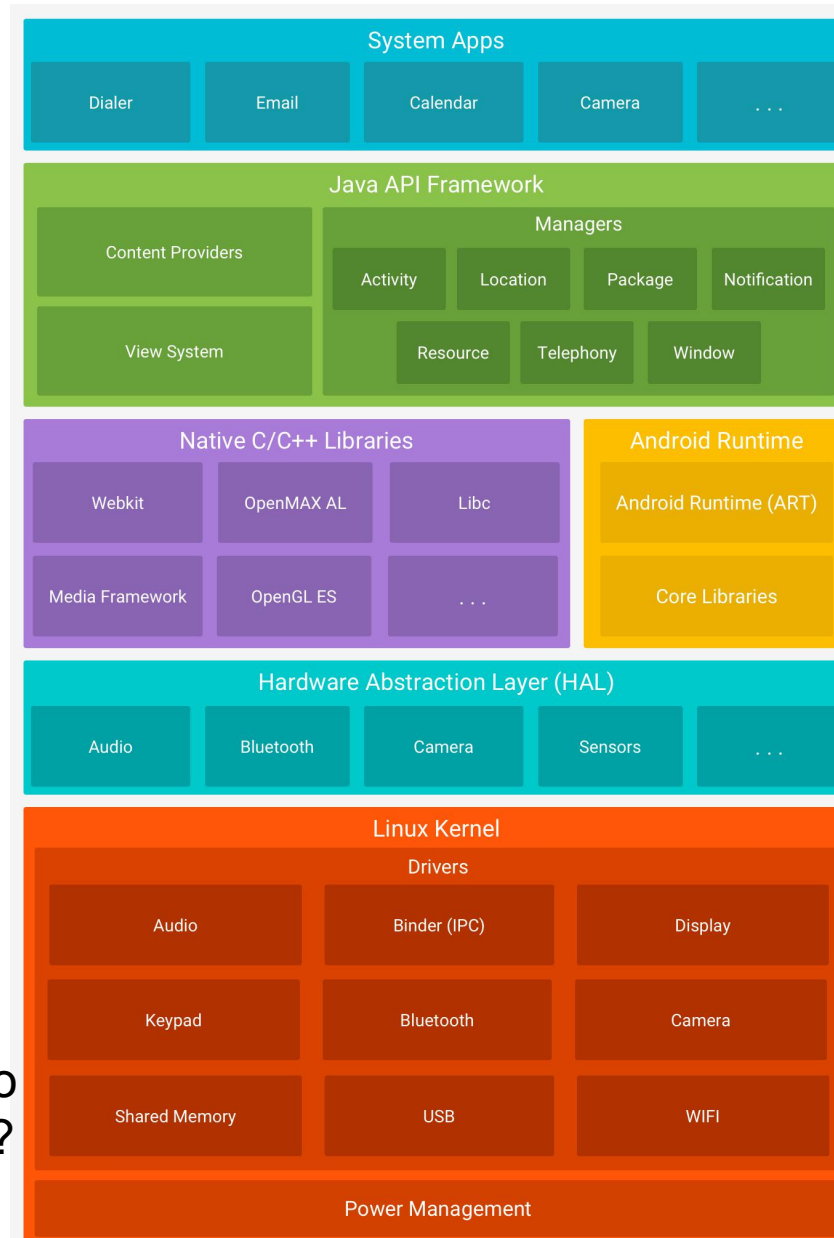
Nome	Versão	Data de lançamento	API	Distribuição	Data do patch de segurança mais recente ^[5]
Android 11	11	8 de setembro de 2020 (5 anos) ^[34]	30	~15.9%	fevereiro de 2024; há 2 anos
Android 12	12	4 de outubro de 2021 (4 anos) ^[35]	31	~12.8%	março de 2025; há 1 ano
Android 12L	12.1	7 de março de 2022 (4 anos) ^[36]	32		
Android 13	13	15 de agosto de 2022 (3 anos) ^[37]	33	~16.8%	agosto de 2025; há 1 ano
Android 14	14	4 de outubro de 2023 (2 anos) ^[38]	34	~27.4%	
Android 15	15	15 de outubro de 2024 (1 ano) ^[39]	35	~4.5%	
Android 16	16	10 de junho de 2025 (1 ano) ^[40]	36	~0.0%	
Android 17	17	16 de junho de 2026 (0 anos) ^[41]	37	~0.0%	

Legenda: ■ Versão antiga ■ Versão mais antiga, ainda mantida ■ Versão estável atual ■ Versão de prévia mais recente ■ Lançamento futuro

API Level

- Ao criar um aplicativo Android, algumas versões do SDK precisam ser definidas:
 - `compileSdk`: versão do Android SDK utilizada para compilar o aplicativo.
 - `minSdk`: versão mínima do Android compatível com o aplicativo.
 - `targetSdk`: versão do Android para a qual o aplicativo foi projetado e testado, indicando ao sistema quais comportamentos e recursos modernos devem ser aplicados.

Arquitetura da plataforma Android



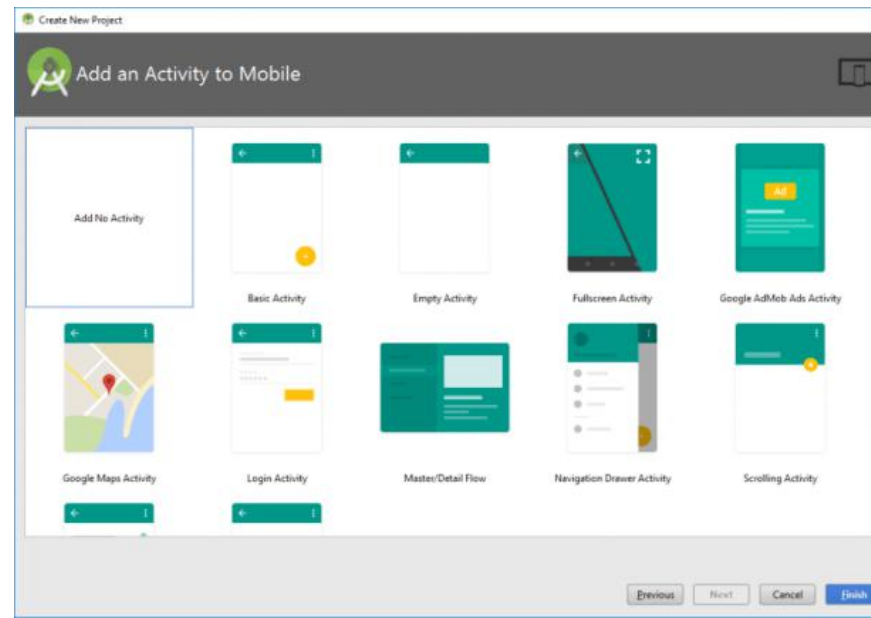
<https://developer.android.com/guide/platform?hl=pt-br>

Arquitetura da plataforma Android

- O Android Runtime (ART) é o ambiente de execução utilizado pelo Android para executar aplicações.
 - O ART substituiu a Dalvik a partir do Android 5.0 (Lollipop).
- O ART introduziu melhorias de desempenho, gerenciamento de memória e coleta de lixo.
- Atualmente, aplicações desenvolvidas em Kotlin ou Java são compiladas para bytecode compatível com o Android, que é convertido para o formato DEX (Dalvik Executable) e executado pelo ART.

Principais componentes do Android

- Activity é um dos principais componentes do Android e representa uma tela ou ponto de interação da aplicação.
 - É responsável por apresentar e gerenciar a interface do usuário (UI).
 - Possui um ciclo de vida, controlado pelo sistema Android.
 - Diferentemente de aplicações Java tradicionais, um aplicativo Android não possui necessariamente um método `main()` como ponto de entrada.



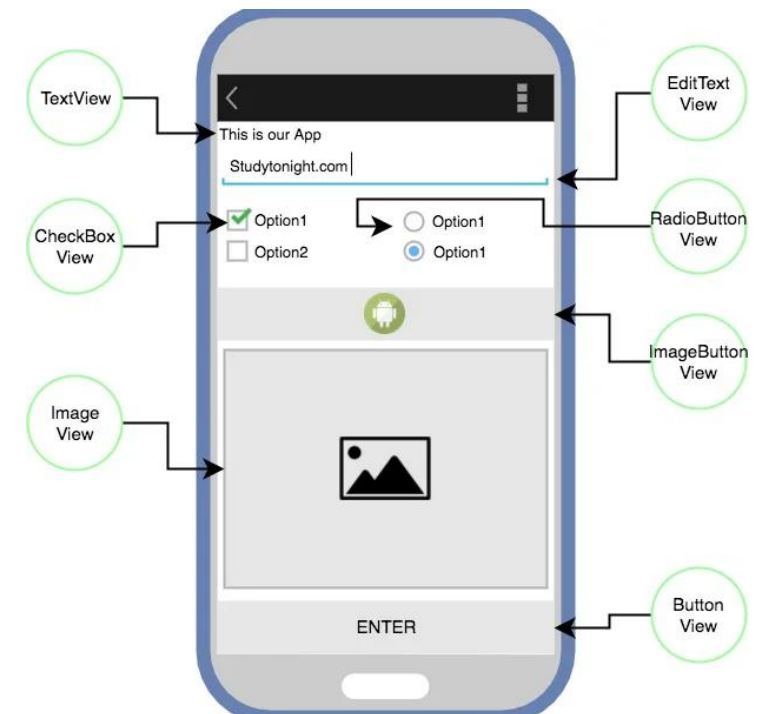
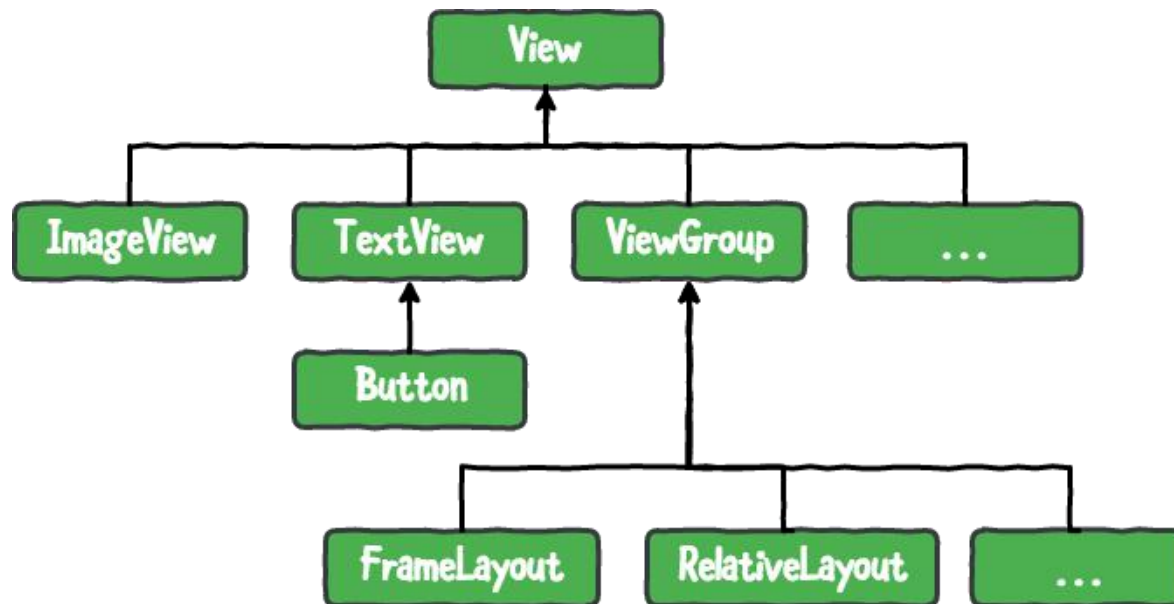
Principais componentes do Android

- Intent é um objeto utilizado para solicitar uma ação ou estabelecer comunicação entre componentes do Android.
- Pode ser utilizado para:
 - Iniciar outra Activity;
 - Iniciar um serviço;
 - Interagir com outros aplicativos.



Principais componentes do Android

- View é o bloco de construção básico da interface de usuário (UI) no Android.
 - Representa um elemento visual da interface.
 - Ocupa uma determinada área da tela.
 - Possui diversas subclasses, como TextView, Button, ImageView e EditText.



Instalação do Android Studio

- Link (instalação): <https://developer.android.com/studio>
- Link (documentação): <https://developer.android.com/docs>



Developers Essenciais ▾ Desenvolver ▾ Mais ▾

Pesquisa /

Português ... ▾ Android Studio Fazer login

ANDROID STUDIO

Fazer download Guias de ambientes de desenvolvimento integrados Gemini no Android Studio Ferramentas e recursos para agentes Prévia do Android Studio Mais ▾

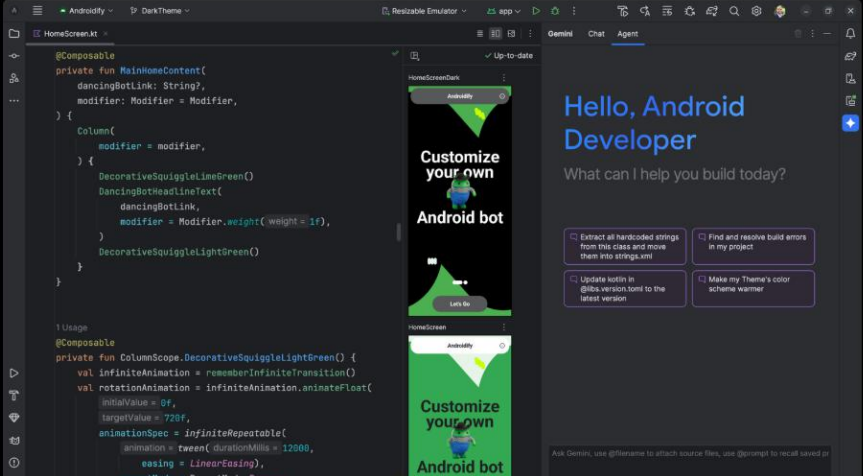
translated by Google O Google usa tecnologia de IA na tradução de conteúdos para seu idioma de preferência. As traduções com IA podem ter erros. [Switch to English](#)

Android Studio

O ambiente de desenvolvimento integrado oficial para desenvolvimento de apps Android agora acelera sua produtividade com o Gemini no Android Studio, seu parceiro de programação com tecnologia de IA.

Fazer o download do Android Studio Quail 3

Leia as notas de lançamento



Instalação do Android Studio

MÓDULO 2



5 ATIVIDADES

Configurar o Android Studio

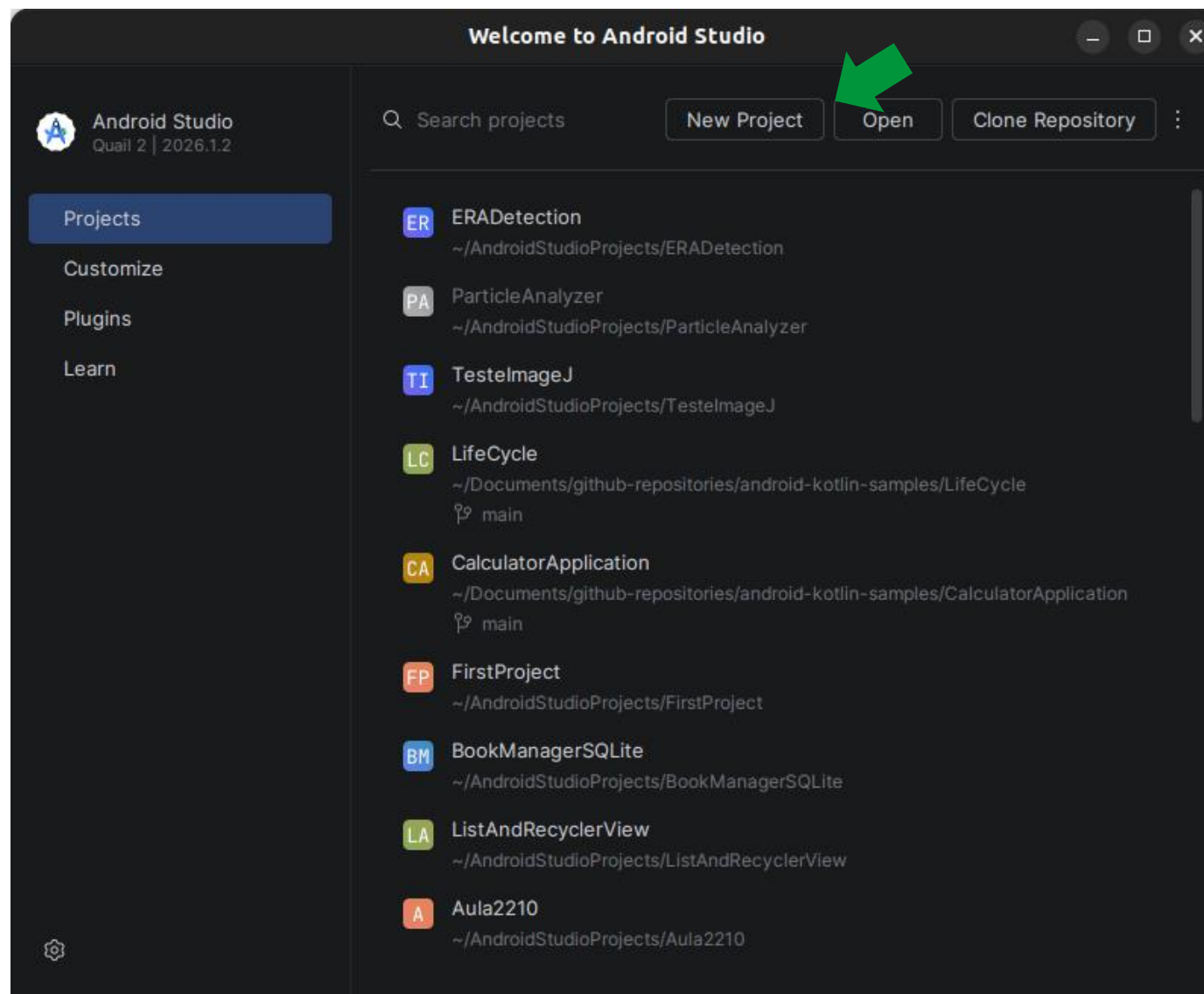
Instale e configure o Android Studio. Crie e execute seu primeiro projeto em um dispositivo ou emulador.

Abril de 2022

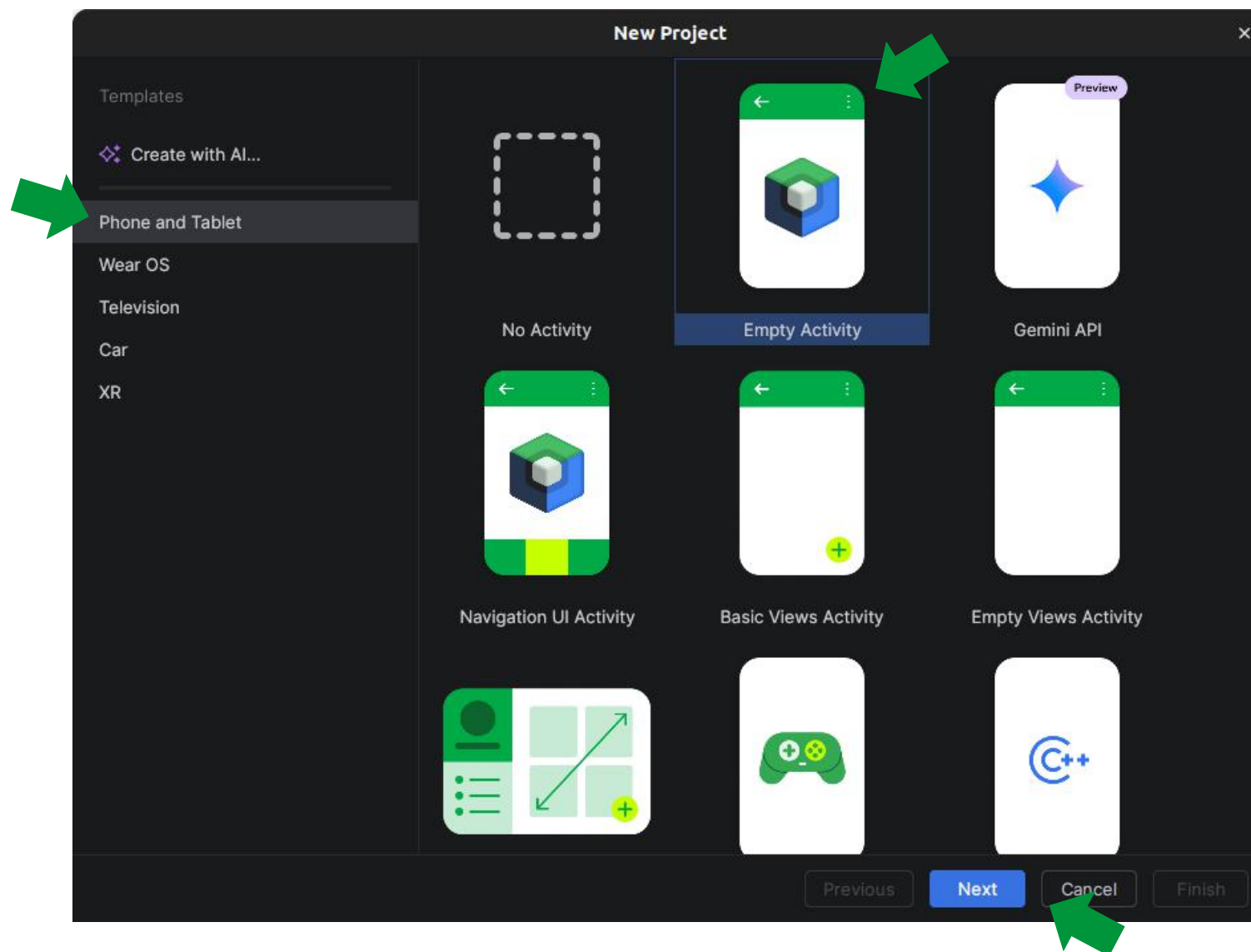
Confira

Link: <https://developer.android.com/courses/android-basics-compose/course>

Criando e executando um primeiro projeto



Criando e executando um primeiro projeto



Criando e executando um primeiro projeto

New Project [X]

Empty Activity

Create a new empty activity with Jetpack Compose

Name: TestApplication

Package name: leite.sampaio.lucas.testapplication

Save location: /home/lucas/AndroidStudioProjects/TestApplication [Folder icon]

Minimum SDK: API 24 ("Nougat"; Android 7.0) [Dropdown arrow]

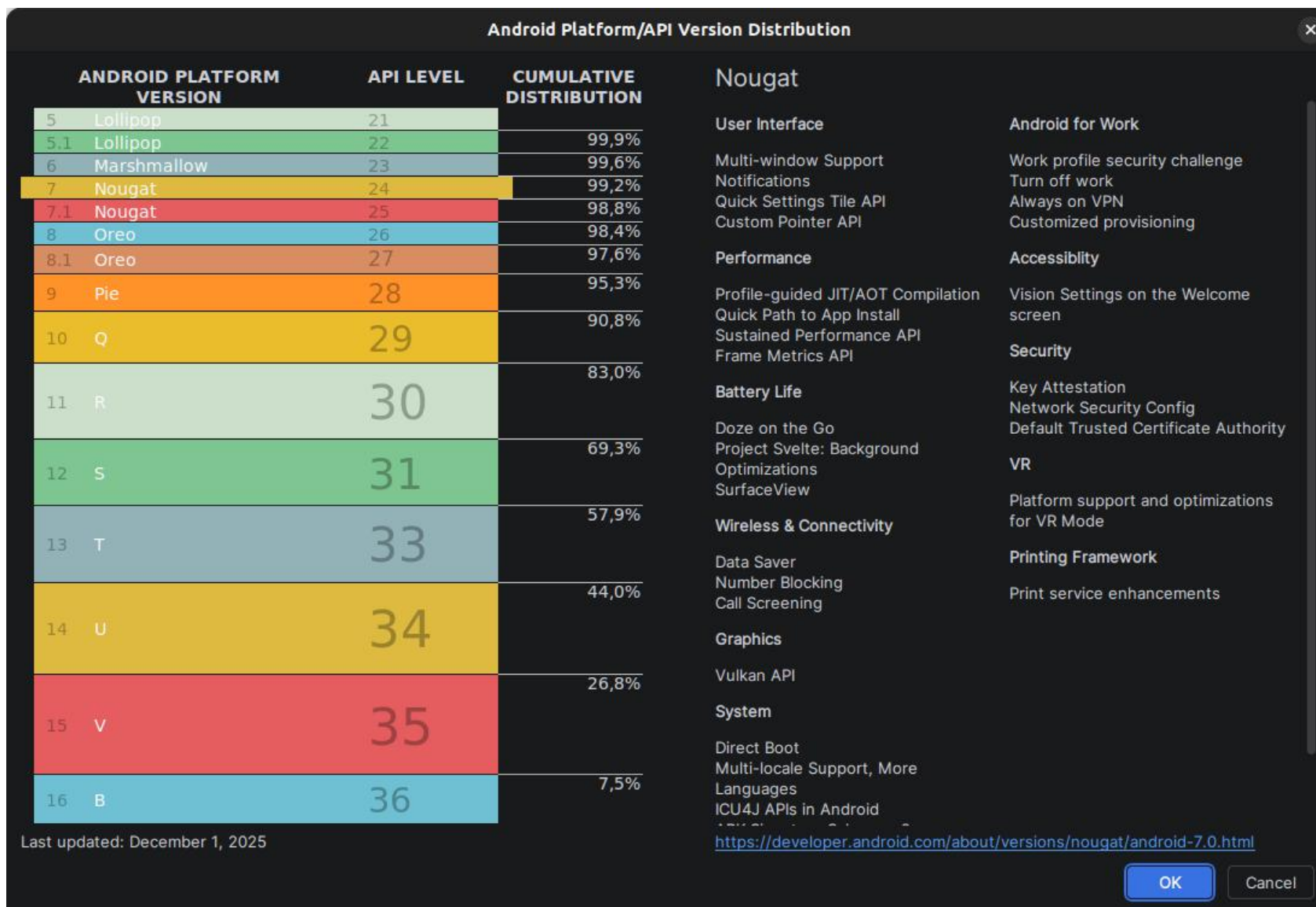
i Your app will run on approximately 99,2% of devices.
[Help me choose](#)

Build configuration language *?*: Kotlin DSL (build.gradle.kts) [Recommended] [Dropdown arrow]

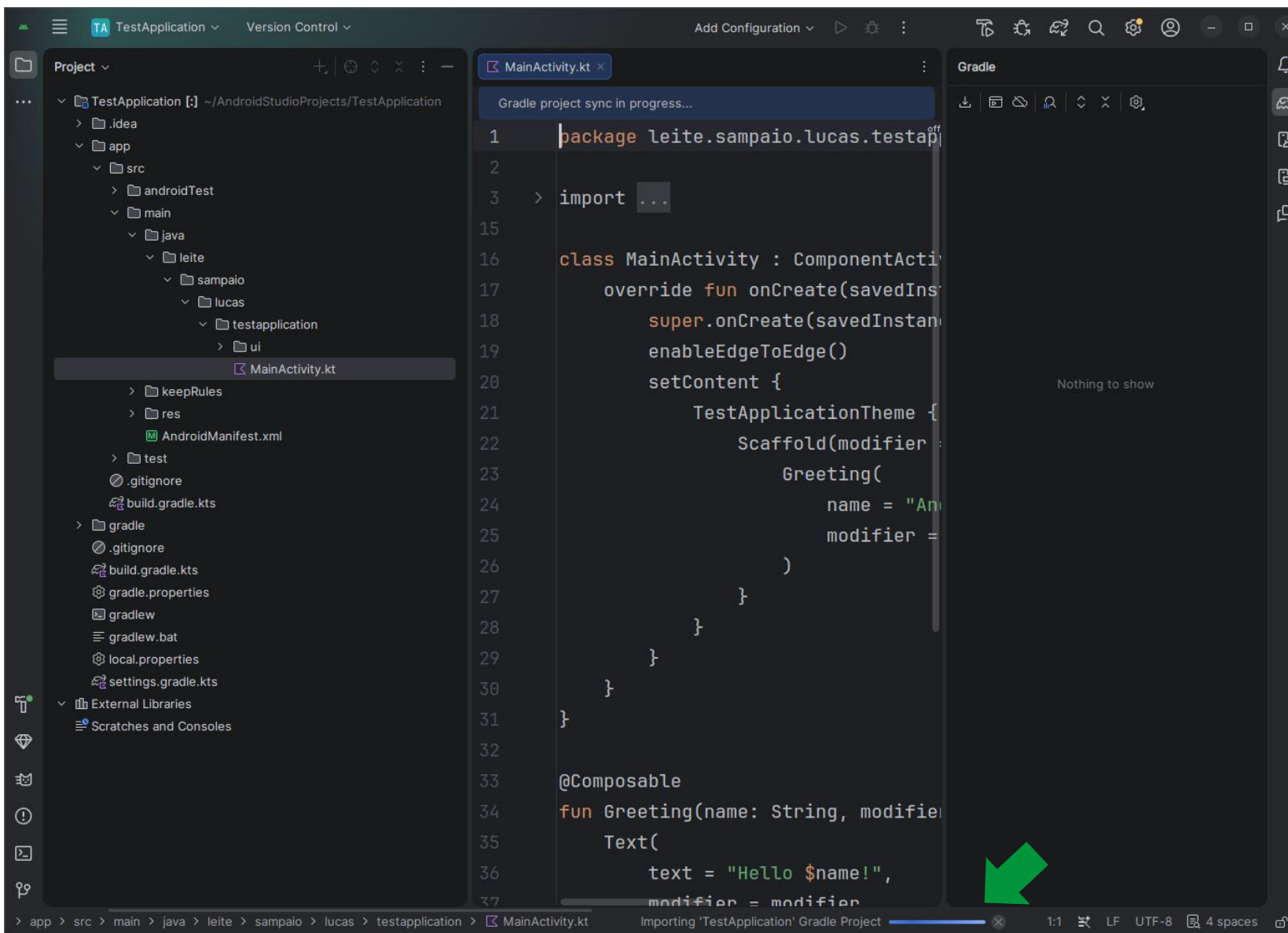
Previous Next Cancel **Finish**



Criando e executando um primeiro projeto

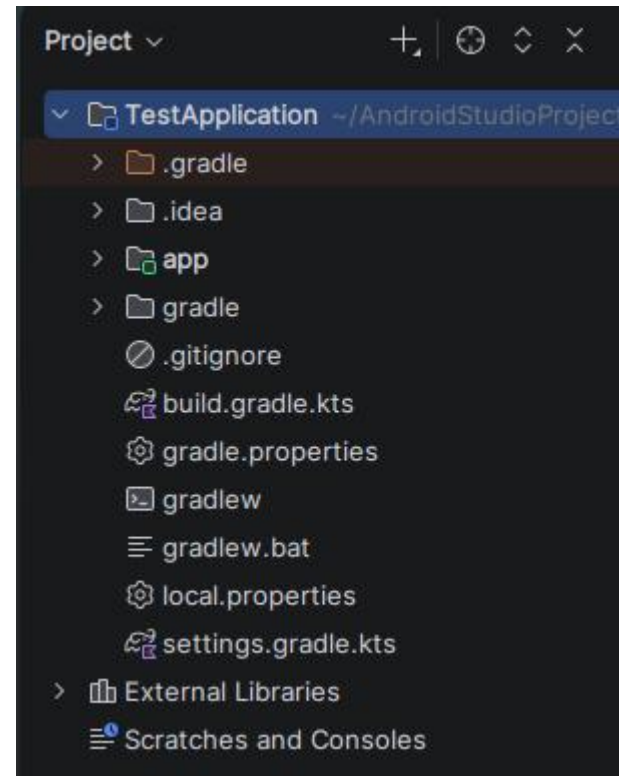
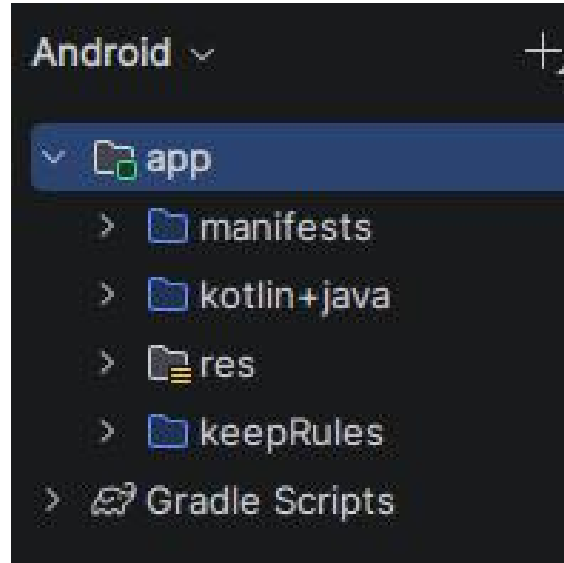


Criando e executando um primeiro projeto



Estrutura de um projeto android

- O Android Studio permite trabalhar com apenas um projeto por instância.
- Android: exibe o projeto de forma simplificada para o desenvolvimento.
- Project: exibe a estrutura real de arquivos e diretórios do projeto.

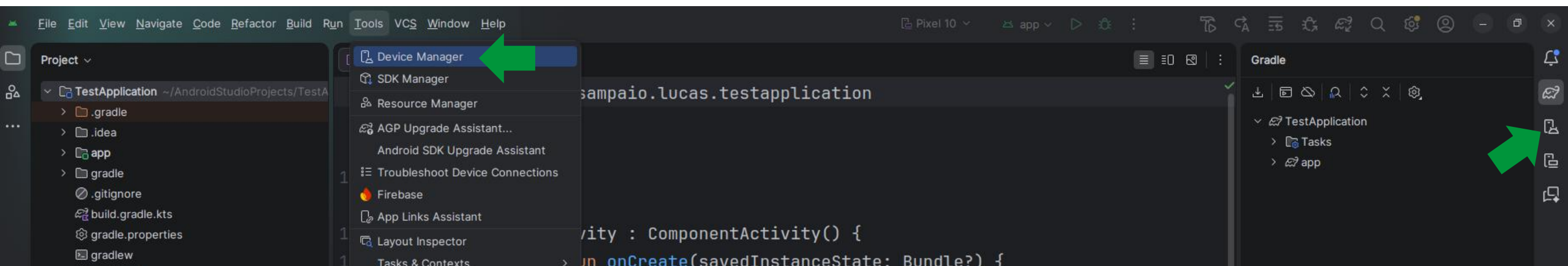


Estrutura de um projeto android

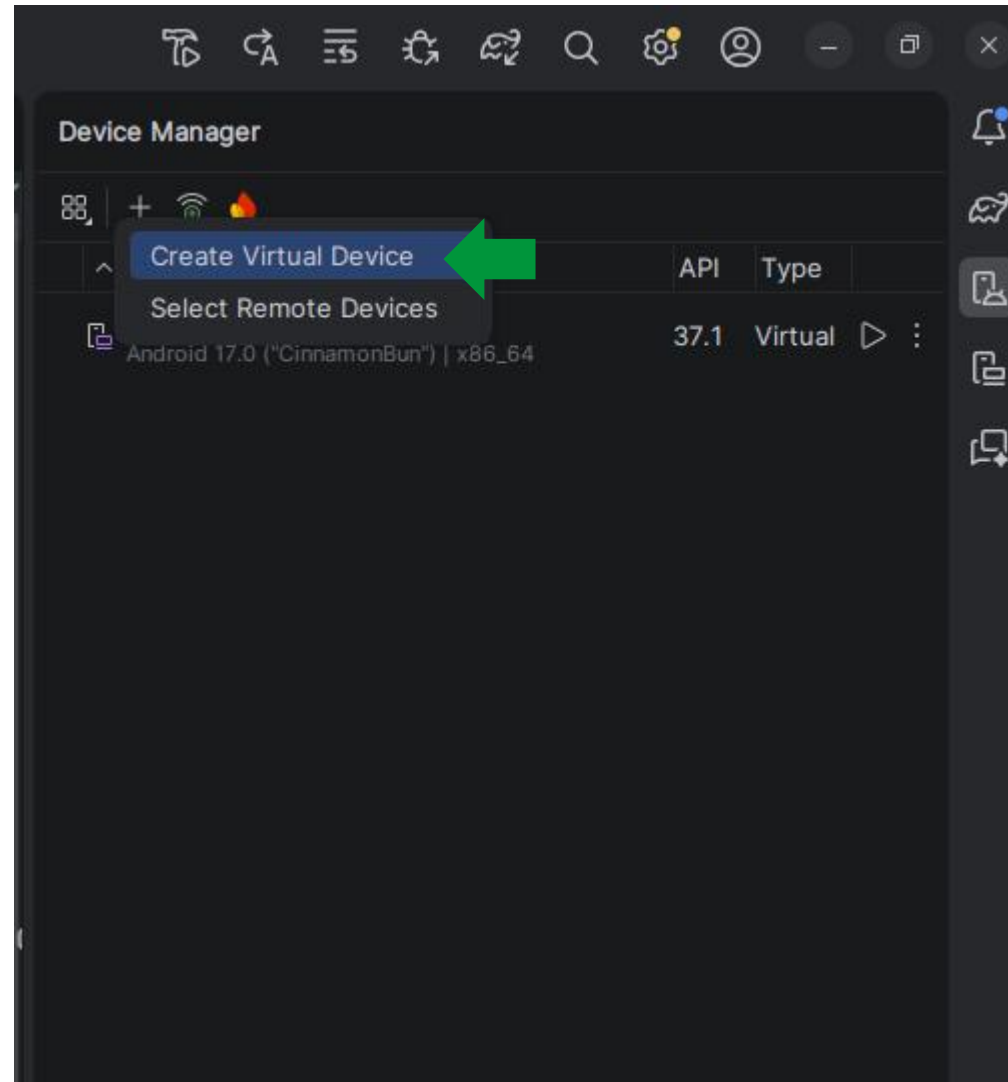
Diretório	Descrição
build	Arquivos gerados durante a compilação do projeto
src/main/java	Código fonte Java e Kotlin
src/main/res	Recursos da aplicação
src/test/java	Código dos testes unitários
res/drawable	Imagens e recursos gráficos
res/mipmap	Ícones do aplicativo
res/layout	Layouts das telas
res/values	Cores, strings e temas
AndroidManifest.xml	Configurações e componentes da aplicação
build.gradle.kts	Configurações de compilação do projeto/módulo

Device Manager

- O Device Manager é a ferramenta do Android Studio utilizada para criar e gerenciar dispositivos virtuais (AVDs).
 - Permite criar emuladores com diferentes modelos de dispositivos e versões do Android.
 - Permite configurar características como tamanho da tela, resolução, memória e API Level.
 - Permite iniciar, parar, editar e excluir dispositivos virtuais.
 - Também permite gerenciar dispositivos físicos conectados ao computador.



Device Manager



Device Manager

Form Factor

☒ Phone

☐ Tablet

☐ Wear OS

☐ Desktop

☐ TV

☐ Automotive

☐ XR

☐ Show obsolete device profiles

Search for a device by name

Name	Play	API	Width	Height	Density
Small Phone		24+	720	1280	320 dpi
Medium Phone		24+	1080	2400	420 dpi
Resizable (Experimental)		34+	1080	2400	420 dpi
Pixel 10 Pro XL		36+	1344	2992	480 dpi
Pixel 10 Pro Fold		36+	2076	2152	390 dpi
Pixel 10 Pro		36+	1280	2856	480 dpi
Pixel 10		36+	1080	2424	420 dpi
Pixel 9a		35+	1080	2424	420 dpi
Pixel 9 Pro XL		35+	1344	2992	480 dpi
Pixel 9 Pro Fold		35+	2076	2152	390 dpi
Pixel 9 Pro		35+	1280	2856	480 dpi
Pixel 9		35+	1080	2424	420 dpi
Pixel 8a		34+	1080	2400	420 dpi
Pixel 8 Pro		34+	1344	2992	480 dpi
Pixel 8		34+	1080	2400	420 dpi
Pixel Fold		34+	2208	1840	420 dpi
Pixel 7a		34+	1080	2400	420 dpi
Pixel 7 Pro		33+	1440	3120	560 dpi
Pixel 7		33+	1080	2400	420 dpi
Pixel 6a		33+	1080	2400	420 dpi
Pixel 6 Pro		31+	1440	3120	560 dpi
Pixel 6		31+	1080	2400	420 dpi

New hardware profile...

Import hardware profile...

Cancel

Previous

Next

Device Manager

Add Device

Configure virtual device

Device

Additional settings

Name

Pixel 10 (2)

Select system image

Use the filters to help find the system image that you prefer. The combination of device profile and system image is only an approximation of the equivalent physical hardware.

API

Services

API 37.1 "CinnamonBun"; Android 17.0

Google Play Store

System Image

API

★ 16 KB Page Size Google Play Intel x86_64 Atom System Image

37.1

☐ Show system images with SDK extensions

☐ Show unsupported system images

Pixel 10

1080 px

2424 px

6,3"

Device

OEM Google

Supported API Levels 36+

System Image

API Level 37.1

Services Google Play

ABI x86_64

Translated ABI arm64-v8a

Screen

Resolution 1080 × 2424

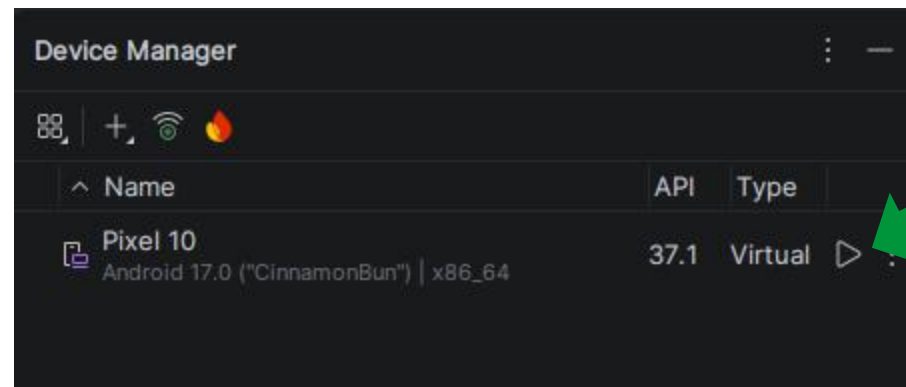
Density 420 dpi

Cancel

Previous

Finish

Device Manager

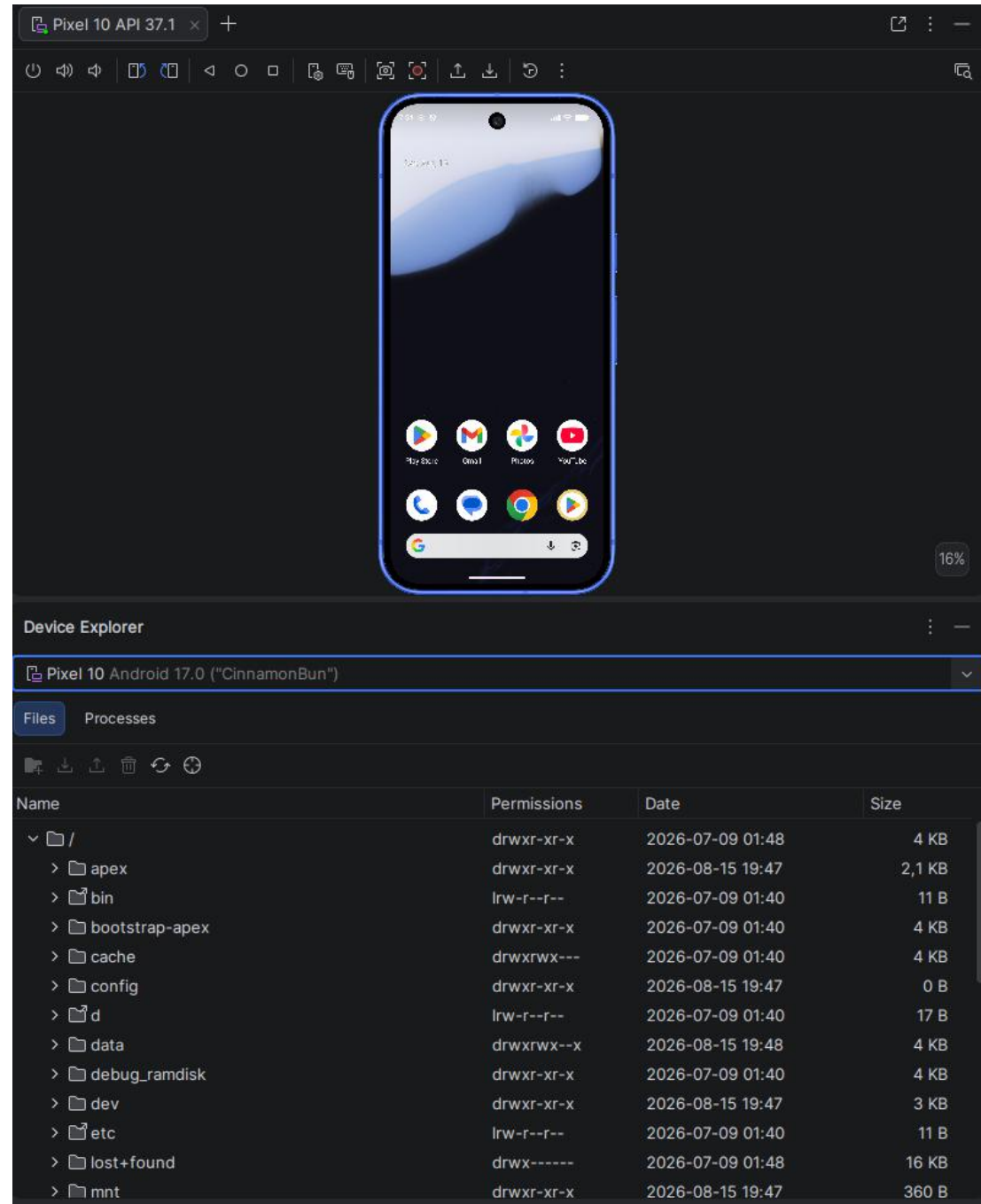


Device Manager

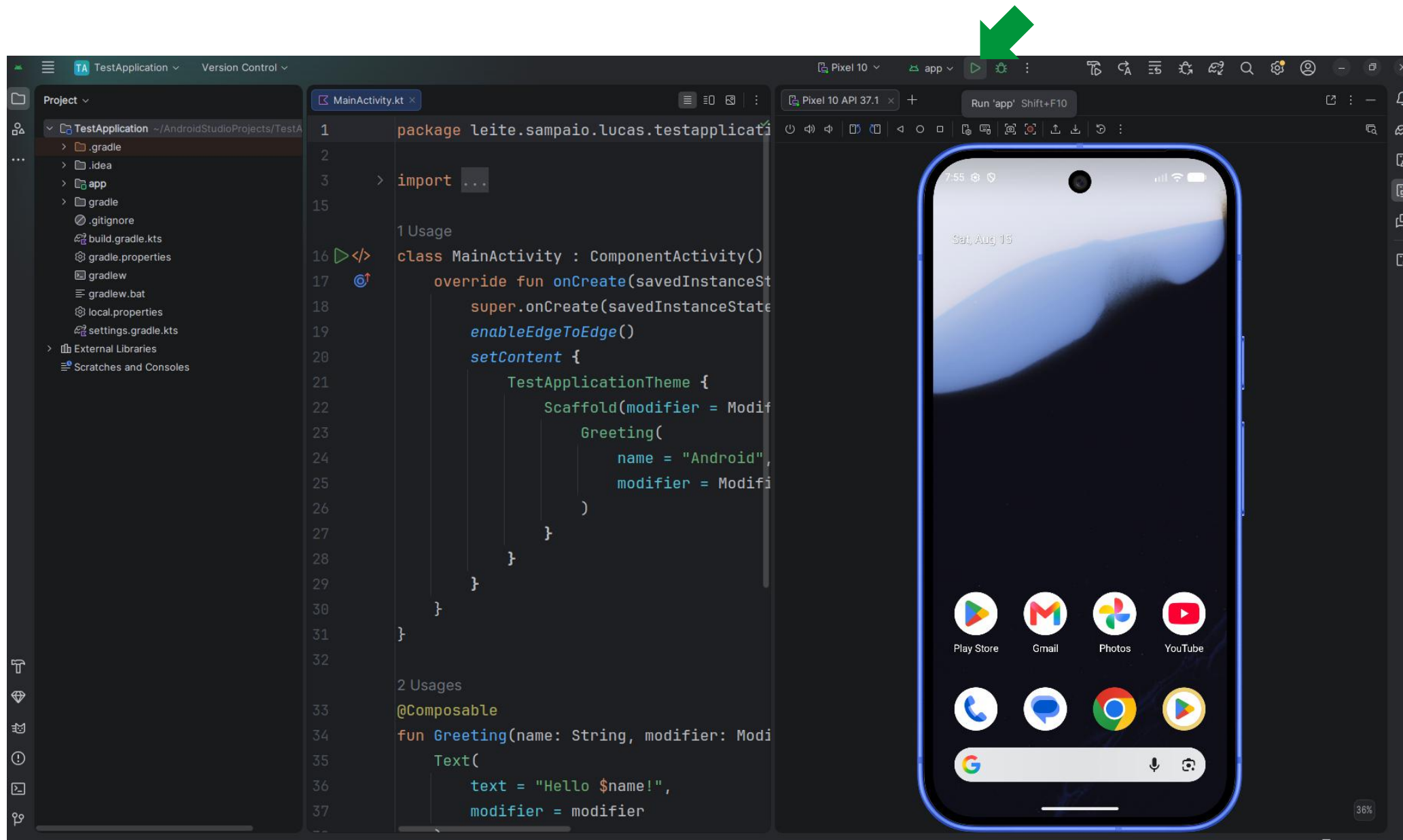


Device Manager

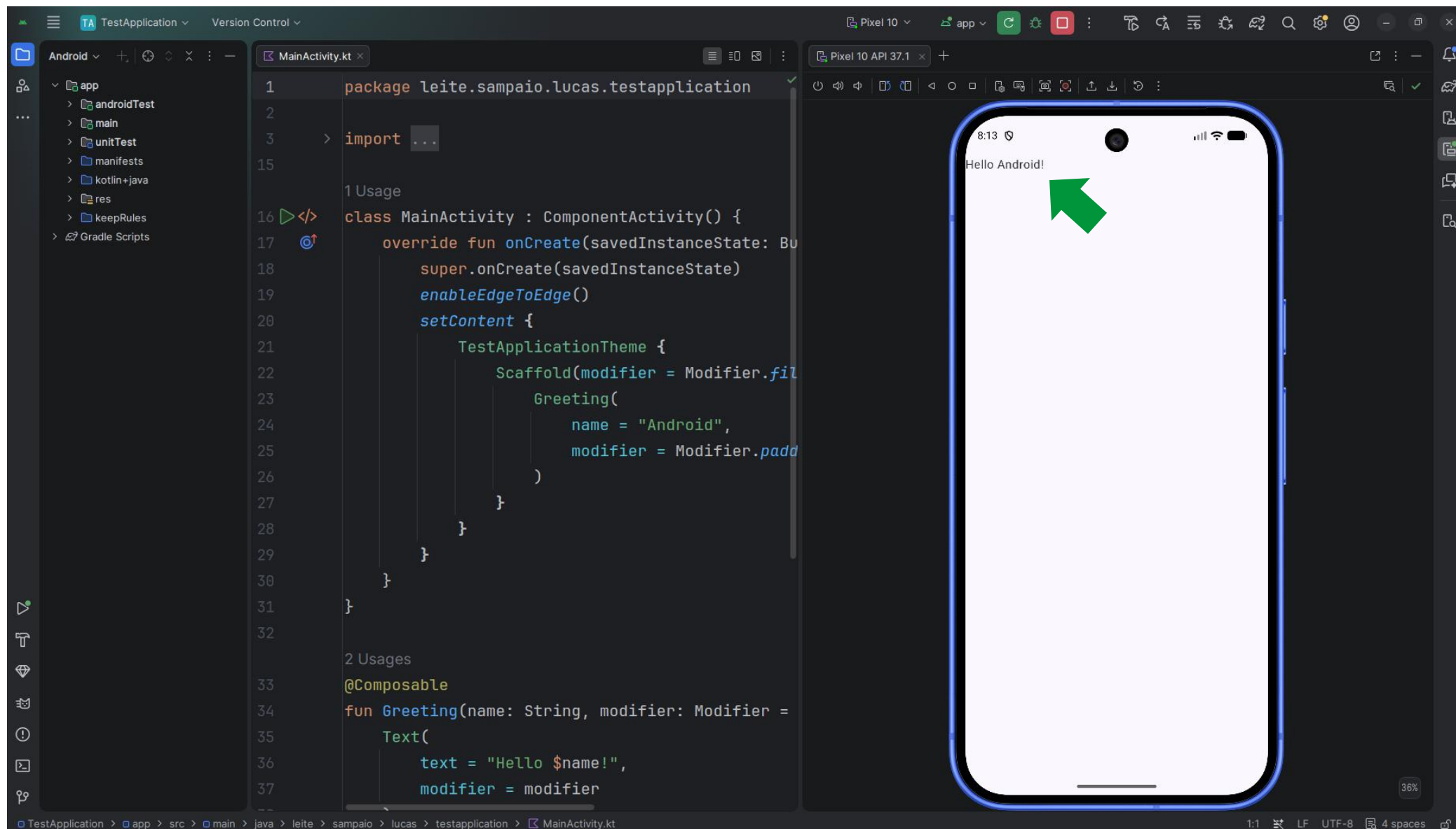
- O Device Explorer permite visualizar e gerenciar os arquivos armazenados em um dispositivo Android conectado ou em um emulador.
 - View → Tool Windows → Device Explorer



Executando o projeto no emulador



Executando o projeto no emulador

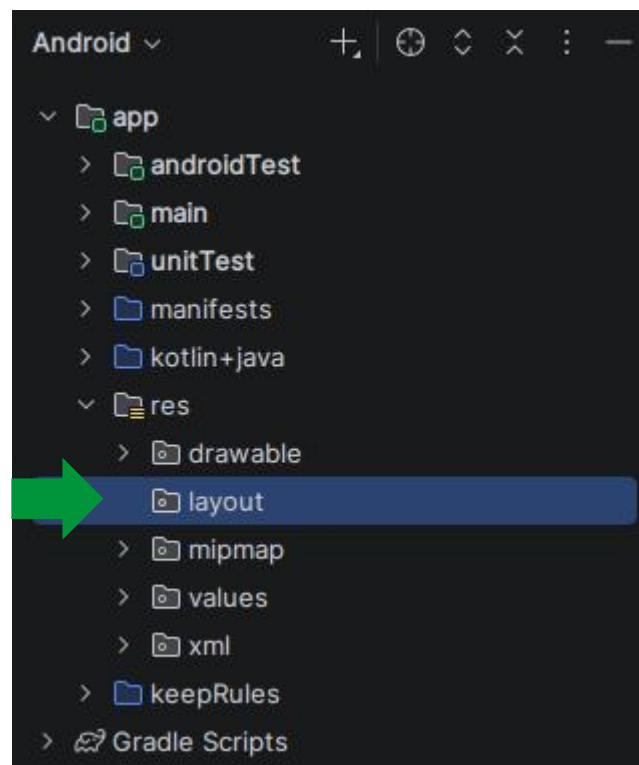


XML x Jetpack Compose

- XML (Views)
 - Interface definida em arquivos XML.
 - Utiliza Views e ViewGroups.
 - Layout separado do código Kotlin.
 - Abordagem tradicional e consolidada.
- Jetpack Compose
 - Interface definida diretamente em Kotlin.
 - Utiliza funções @Composable.
 - Código e interface ficam no mesmo ambiente.

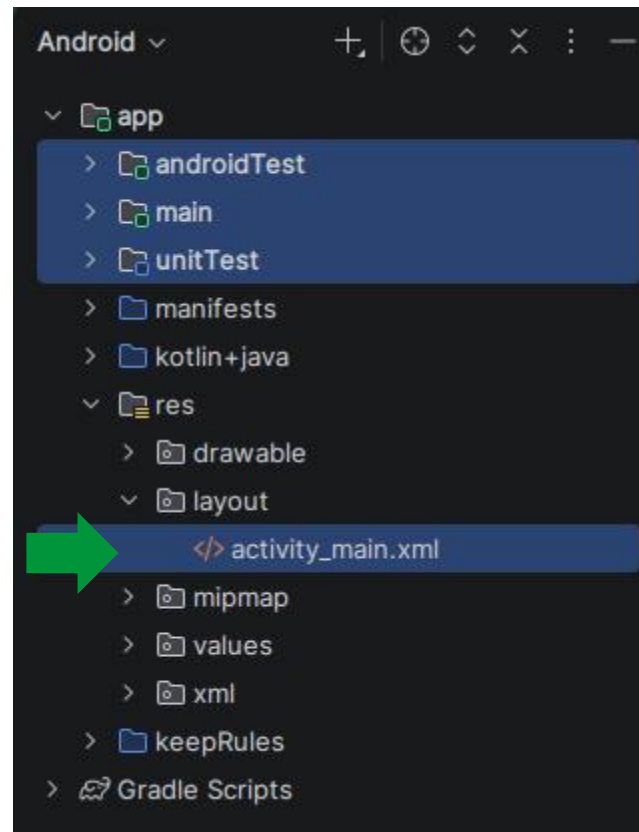
Criando um layout

- res → New → Directory → Name = layout

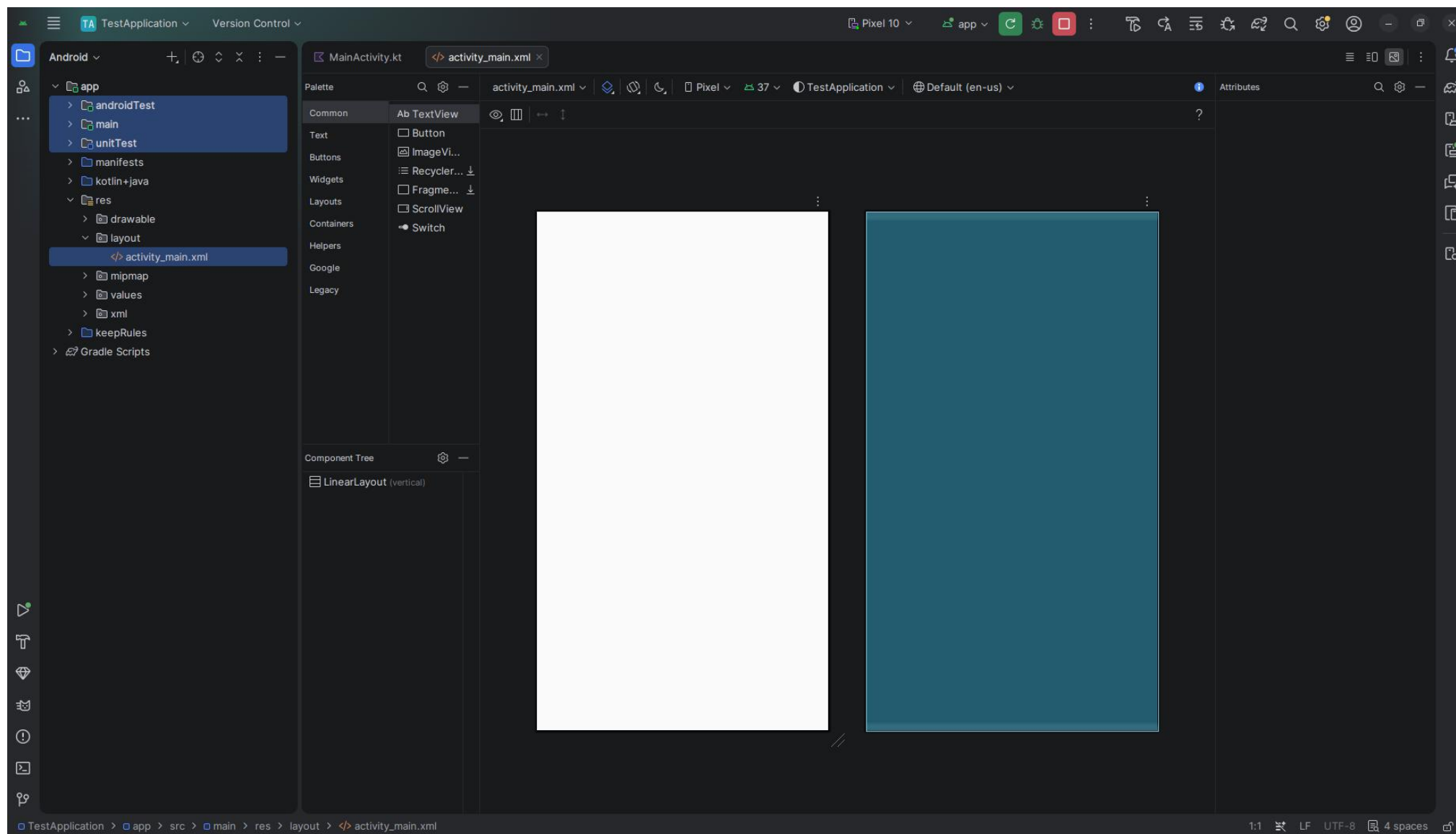


Criando um layout

- layout → New → Layout Resource File → File name = activity_main

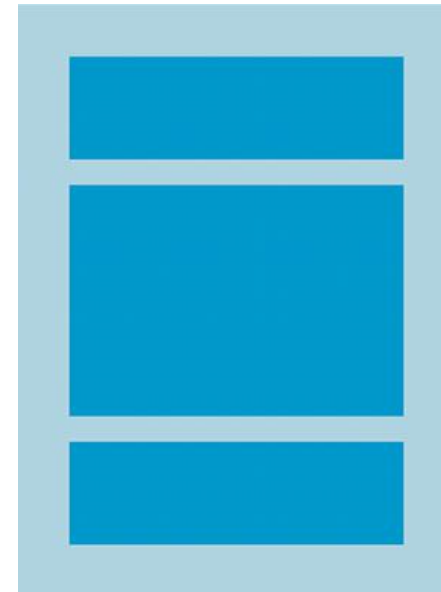


Criando um layout



LinearLayout

- LinearLayout é um ViewGroup que organiza seus elementos filhos em uma única direção: vertical ou horizontal.

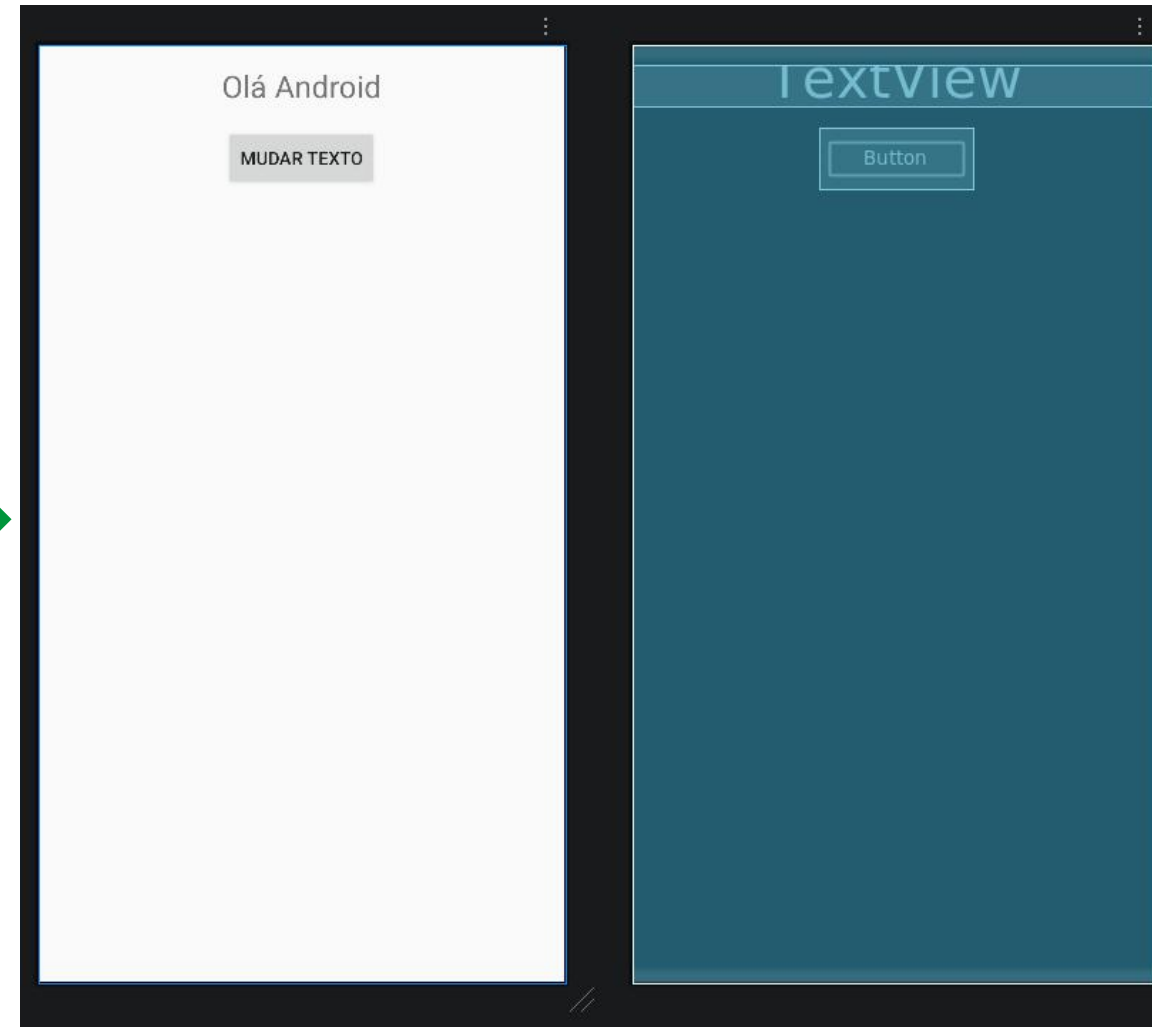


Prática 1 – Criando Views e tratando o clique de um botão

1. Criar um TextView.
2. Criar um Button.
3. Tratar o evento de clique do botão.
4. Alterar o texto do TextView ao clicar no botão.

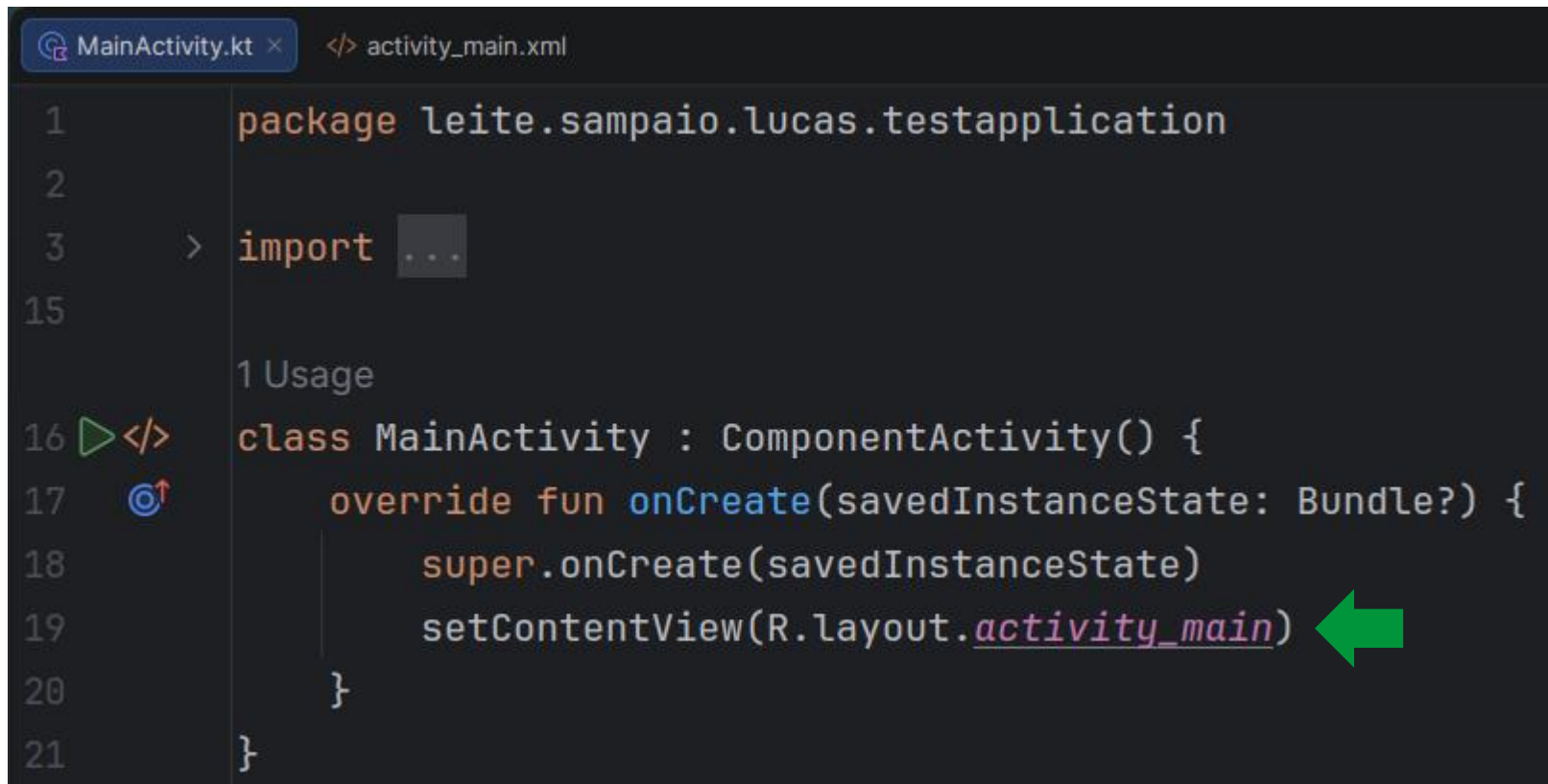
Prática 1 – Criando Views e tratando o clique de um botão

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:orientation="vertical"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent">
6
7     <TextView
8         android:id="@+id/textView"
9         android:layout_width="match_parent"
10        android:layout_height="wrap_content"
11        android:layout_marginTop="16dp"
12        android:textSize="24sp"
13        android:gravity="center"
14        android:text="@string/hello_android" />
15
16    <Button
17        android:id="@+id/button"
18        android:layout_width="wrap_content"
19        android:layout_height="wrap_content"
20        android:layout_gravity="center"
21        android:layout_marginTop="16dp"
22        android:text="@string/change_text" />
23 </LinearLayout>
```



Prática 1 – Criando Views e tratando o clique de um botão

- Modificando a Activity: removendo o código do Jetpack Compose e configurando o layout XML.



```
1 package leite.sampaio.lucas.testapplication
2
3 > import ...
15
16 </> class MainActivity : ComponentActivity() {
17     override fun onCreate(savedInstanceState: Bundle?) {
18         super.onCreate(savedInstanceState)
19         setContentView(R.layout.activity_main)
20     }
21 }
```

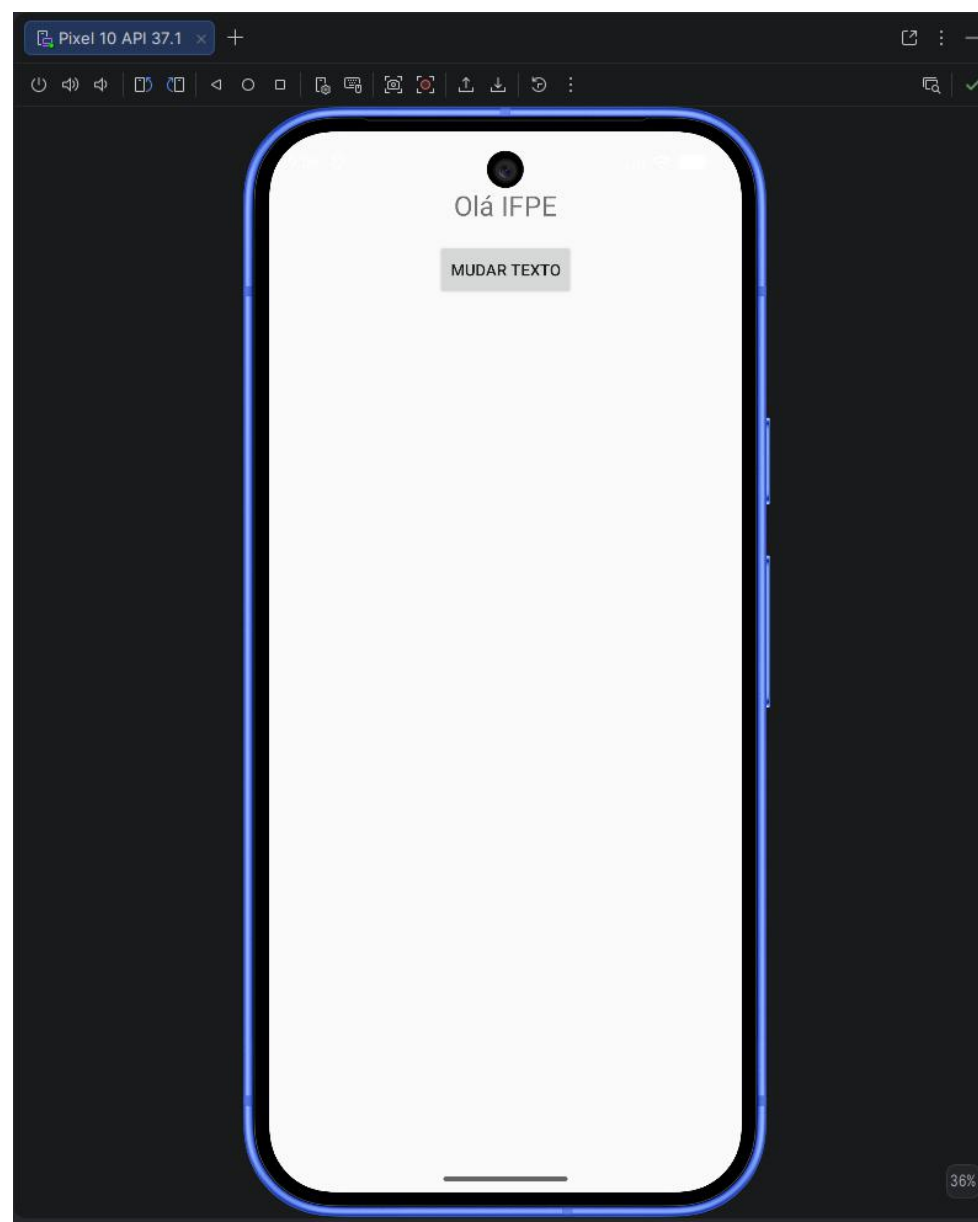
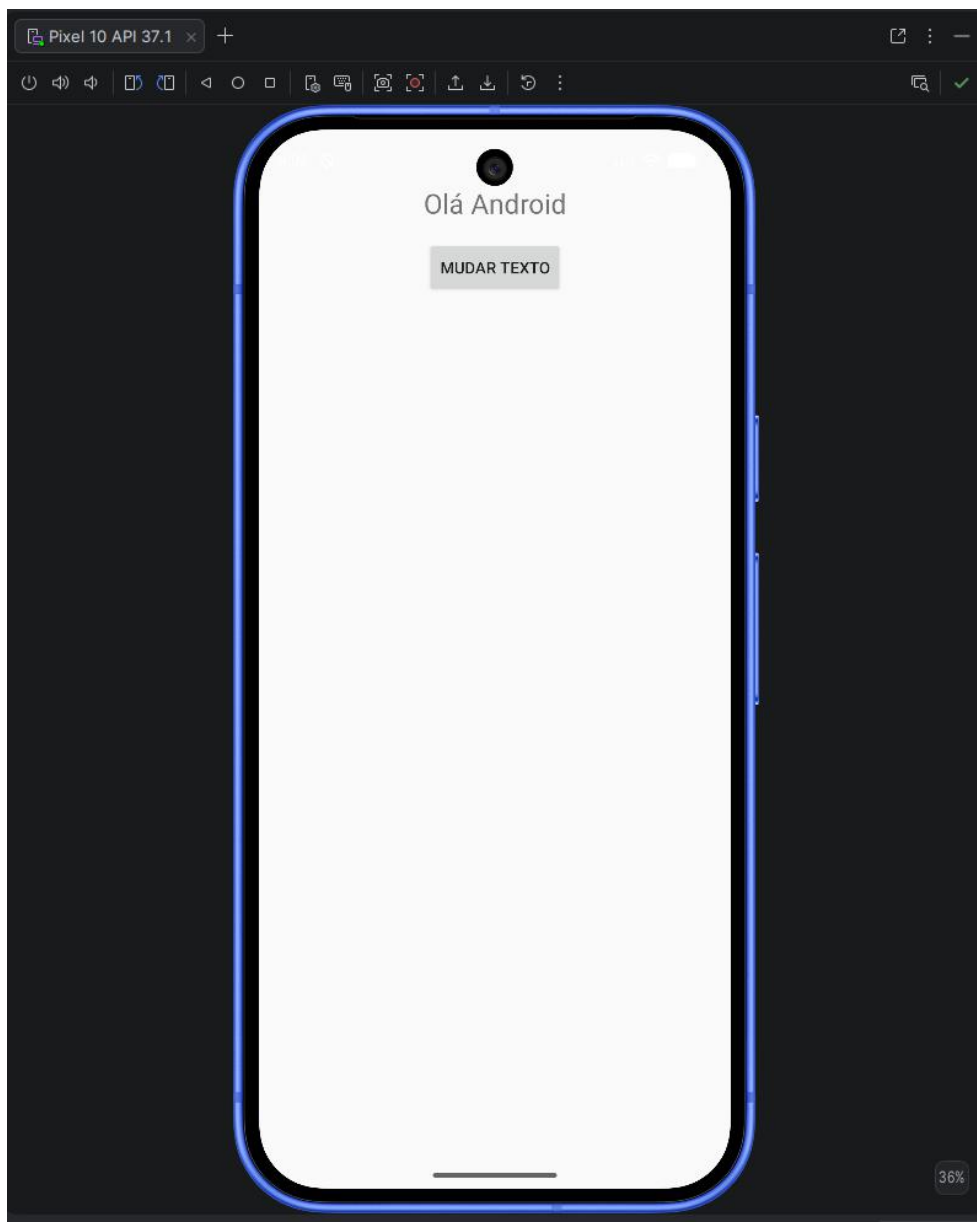
Prática 1 – Criando Views e tratando o clique de um botão

- Recuperando componentes para a construção da lógica
 - O método `findViewById()` permite obter uma referência a uma View a partir do ID definido no layout XML, possibilitando manipulá-la no código Kotlin.

```
MainActivity.kt x  </> strings.xml  </> activity_main.xml
1  package leite.sampaio.lucas.testapplication
2
3  > import ...
17
1 Usage
18 </> class MainActivity : AppCompatActivity() {
19  @<?> override fun onCreate(savedInstanceState: Bundle?) {
20      super.onCreate(savedInstanceState)
21      setContentView(R.layout.activity_main)
22
23      val label: TextView = findViewById(R.id.textView)
24      val button: Button = findViewById(R.id.button)
25
26      button.setOnClickListener {
27          label.text = getString(resId = R.string.label_changed)
28      }
29  }
30 }
```

```
MainActivity.kt  </> strings.xml x  </> activity_main.xml
Edit translations for all locales in the translations editor.
1  <resources>
2      <string name="app_name">TestApplication</string>
3      <string name="hello_android">Olá Android</string>
4      <string name="change_text">Mudar texto</string>
5      <string name="label_changed">Olá IFPE</string>
6
7  </resources>
```

Prática 1 – Criando Views e tratando o clique de um botão



Prática 1 – Criando Views e tratando o clique de um botão



Observação: devido ao comportamento Edge-to-Edge em versões recentes do Android, foi necessário ajustar o `marginTop` para evitar que o conteúdo fique sob a barra de status.

Prática 1 – Criando Views e tratando o clique de um botão

- O View Binding gera automaticamente uma classe para cada arquivo de layout XML, permitindo acessar diretamente as Views do layout pelo código Kotlin.
 - Evita o uso de findViewById().
 - Oferece acesso seguro e tipado aos componentes da interface.
 - É necessário habilitar o View Binding no build.gradle.kts do módulo da aplicação.

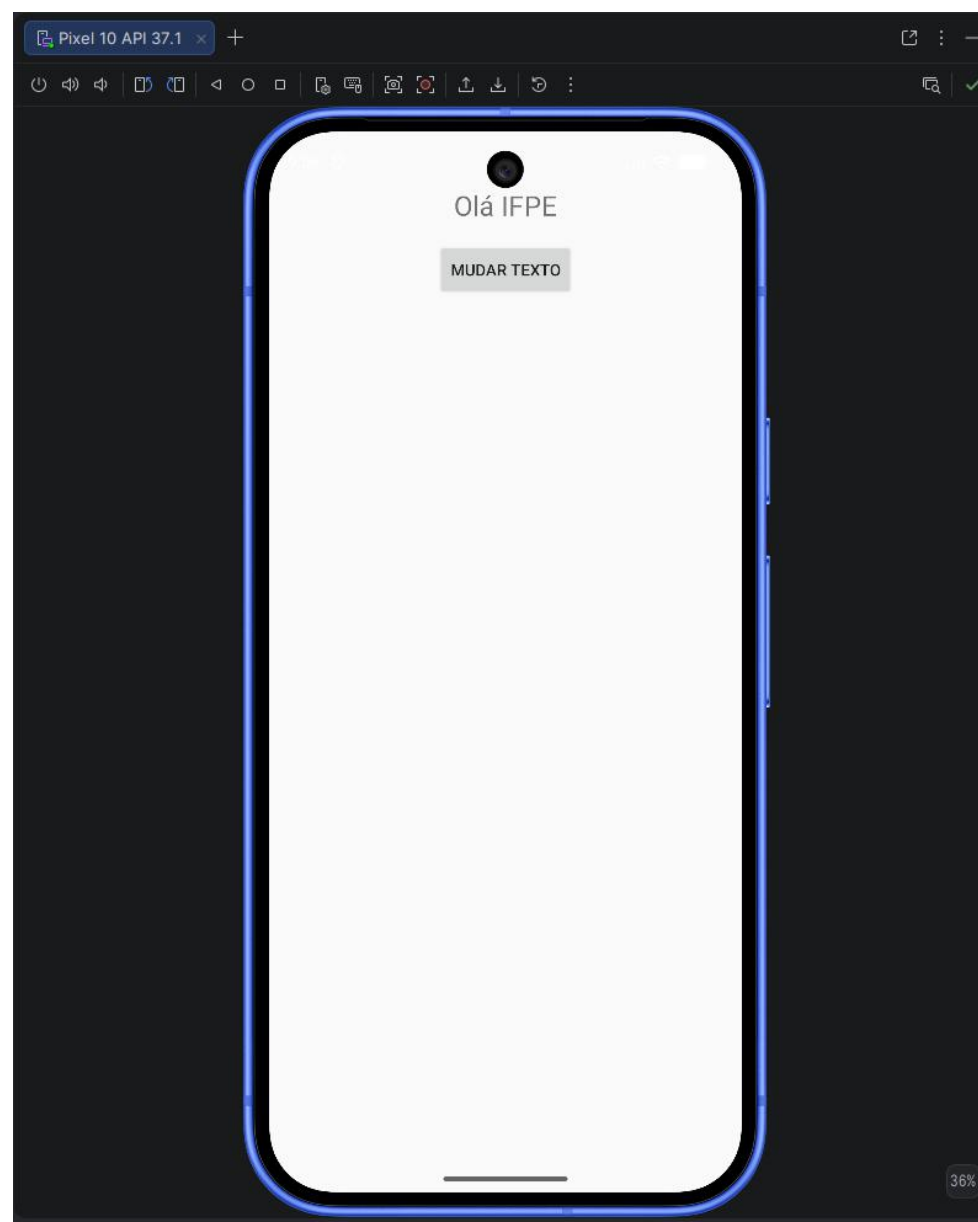
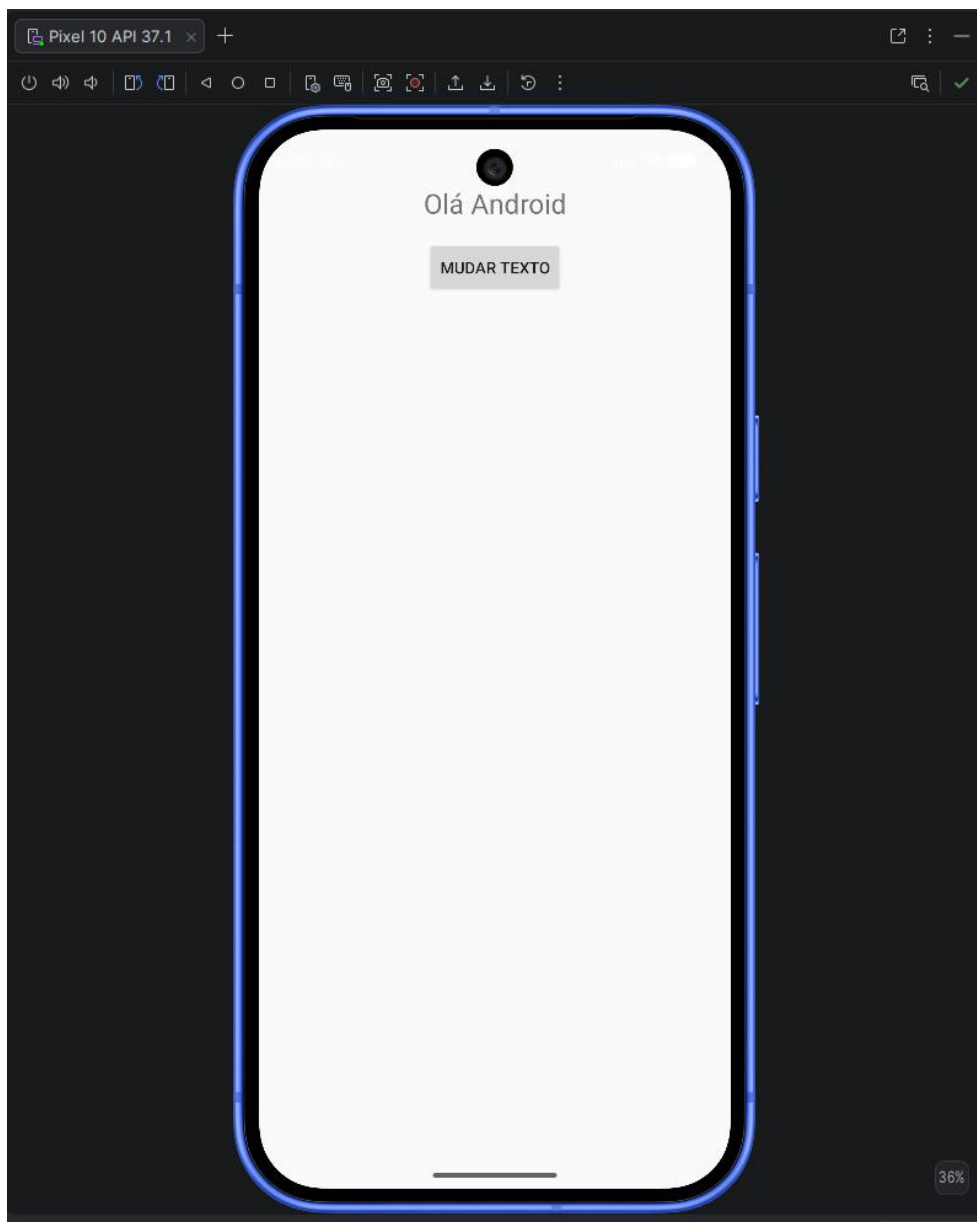
```
viewBinding{  
    enable = true  
}
```

Prática 1 – Criando Views e tratando o clique de um botão

- Recuperando os componentes para construção de lógica:

```
MainActivity.kt x build.gradle.kts (:app) </> strings.xml </> activity_main.xml
1 package leite.sampaio.lucas.testapplication
2
3 > import ...
18
1 Usage
19 </> class MainActivity : ComponentActivity() {
20
21 4 Usages
22 private lateinit var binding: ActivityMainBinding
23
24 override fun onCreate(savedInstanceState: Bundle?) {
25     super.onCreate(savedInstanceState)
26     binding = ActivityMainBinding.inflate(layoutInflater)
27     setContentView(binding.root)
28
29     binding.button.setOnClickListener {
30         binding.textView.text = getString(resId = R.string.label_changed)
31     }
32 }
```

Prática 1 – Criando Views e tratando o clique de um botão



Exercício: Criando uma calculadora simples

1. Crie dois EditText para receber dois valores reais.
2. Crie quatro botões para as operações de soma, subtração, multiplicação e divisão.
3. Ao clicar em um botão, um TextView deverá apresentar o resultado da operação.



Exercício: Criando uma calculadora simples

1. Crie dois EditText para receber dois valores reais.
2. Crie quatro botões para as operações de soma, subtração, multiplicação e divisão.
3. Ao clicar em um botão, um TextView deverá apresentar o resultado da operação.



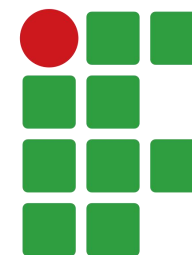
Dica: `val value = binding.edtNum.text.toString().toDoubleOrNull() ?: 0.0`

Dúvidas



PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco